CEA PHD DAY · THESIS PROGRESS · 18 JUNE 2026

Scenario-Based Testing for Autonomous Systems

Improving safety & reliability, in compliance with standards.

Proving autonomous systems are safe before we trust them on the road.

Borhaneddine Hamadou

PhD student · CEA List · Université Paris-Saclay

Supervisors: N. Matragas · C. Mraidha

cea list

THE MOST IMPORTANT WORD IN THIS TALK

Safety ≠ Security

SECURITY



Protecting the system from the world.

world ⟶ system

Can someone make the system do something bad?

SAFETY



Protecting the world from the system.

system ⟶ world

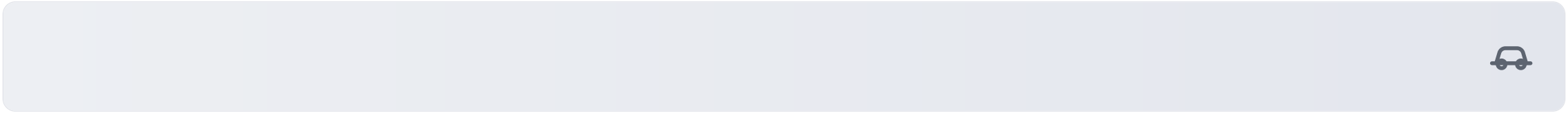
Can the system do something bad all by itself?

This work is about SAFETY.

Proving an autonomous car is safe by driving would take

billions of km

For every software version. Economically and physically impossible.



So how do we ever say a self-driving car is ”safe enough”?

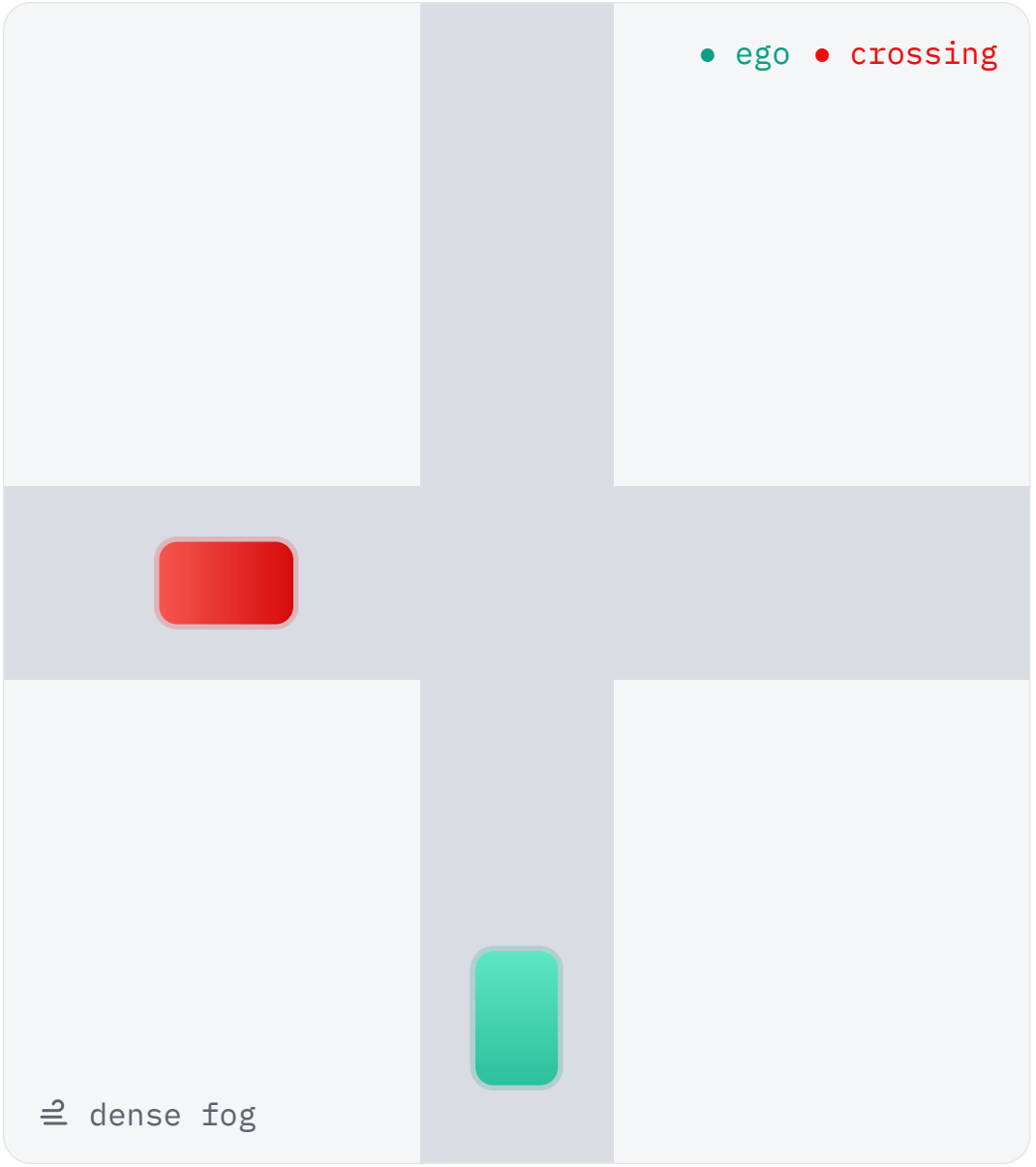
THE FIELD'S ANSWER

Scenario-Based Testing

Instead of driving everywhere, we recreate representative **situations** ("scenarios") in simulation, and check how the system behaves.

EXAMPLE SCENARIO

A car approaches a junction; another vehicle cuts across its path, in dense fog.



Increasingly **required by safety standards**: ISO 21448 / SOTIF, ISO 34503.

BUT SCENARIO-BASED TESTING HAS GAPS

Three open problems

01

What to test?

Which scenarios, out of millions, are worth running?

They must be realistic, diverse and critical, all at once.

02

How much is enough?

When can we honestly stop testing?

No principled "are we done?" answer exists today.

03

How to keep up?

The software changes every day. Re-test everything, every time?

Full re-testing is far too expensive.

Published · ISSRE 2025 Doctoral Symposium

Hypothesis & vision

Scenario parameters are not independent: they have causal dependencies. Model them, group parameters by their **effect**, and reason about which situations are dangerous, so we can generate tests that are more realistic, more effective, and that explain why a failure happens.

Bayesian Networks

Realism: respect how conditions co-occur in the real world.

Capability abstraction

Tractable coverage: group raw parameters by what they do.

Rarity-aware selection

Criticality: focus on the rare, dangerous tail.

Under review · ISSRE main research track

BayScen: what to test

Don't test every weather combination. Ask what each condition **does** to the car, and collapse millions of combinations into 3 capabilities.

millions of raw conditions

↓

Sensor perception

can it **see**?

fog · rain · glare · dark

clear — severe

Surface traction

can it **grip**?

wet · ice · low friction

clear — severe

Lateral stability

can it **hold its line**?

crosswind

clear — severe

Then: make scenarios **realistic**, learned from 43,000+ real weather observations, and focus on the rare, dangerous ones.

Under review · ISSRE 2026

BayScen: how it works

Four steps turn raw weather data into a smart, realistic test suite.

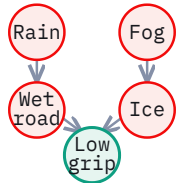
INPUT

- Weather data — 43,000+ obs.
- Operating domain (ODD)
- Situation — e.g. junction

CORE IDEA

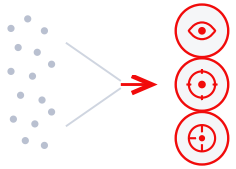
Test by what conditions **do** — see · grip · steer

1 Model



How conditions relate: rain ⟶ wet roads.

2 Abstract



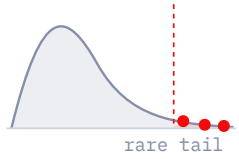
Group by effect: see · grip · steer.

3 Cover



Every combination tested once.

4 Generate



Rare + realistic picks.

OUTPUT



A test suite — **complete**, **realistic**, focused on the **rare**.

More crashes ≠ more useful

~95%
impossible



Adversarial
"crashes found"

~8×
more plausible



BayScen
crashes that are
real

teal = failures that could actually happen on the road

91–95% of adversarial crashes are **physically impossible**: simulation artefacts, not real risks.

BayScen's critical scenarios are **~8× more likely** to be physically plausible, plus the **highest real-world coverage** of any method.

Bonus: testing **by capability** shows why each design fails: one is blinded by fog, another loses grip on slippery roads.

Under review · ASE 2026

PWCC: how much is enough

A test suite can look complete on every metric, and still miss the conditions the car will actually meet. So when can we really stop?

% OF REAL-WORLD RISK ADEQUATELY TESTED



Weight every situation by **how likely it is in real operation**, then declare testing adequate only when untested risk drops below a chosen safety threshold.

from "we ran **many tests**"

to "we covered **enough real-world risk** to stop, with X% confidence."

Under review · ASE 2026

PWCC: how it works

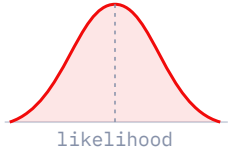
Four steps turn "we ran some tests" into a defensible stop / keep-going decision.

INPUTS

- **Operating data** — what occurs
- **Tests run** — + run time
- **Safety settings** — risk · stop

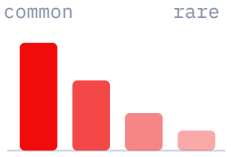
CORE IDEA

Weight everything by **real-world likelihood**



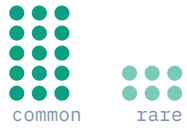
1 Model

Likelihood of each condition.



2 Weight

Importance = how often it happens.



3 Require

More tests where it is common.



4 Decide

Stop when risk < threshold.

OUTPUT



coverage score

STOP ✓

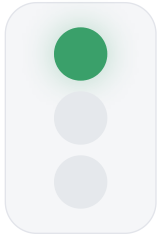
KEEP TESTING

A clear **stop / keep-testing** verdict.

01 GENERATE → 02 MEASURE PWCC → 03 MAINTAIN

Guarding against false sufficiency

Declaring testing done while real failures are still hiding: the most dangerous mistake in the field.

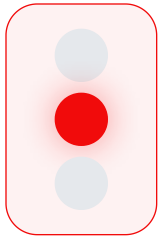


Standard metric

"Done!" ✓

...while hiding **97%+** of unseen failures.

VS



PWCC

"Not yet" (correctly)

Zero false "all-clear" verdicts.

+0.981

strongest predictor of real failures (4 datasets, 7 baselines)

6× fewer

false "all-clear" verdicts than the standard metric

≤ 0.006

score barely moves — a stable, trustworthy signal

The decisive ingredient is **weighting by real-world probability**: without it, you certify safety exactly where the system is most exposed.

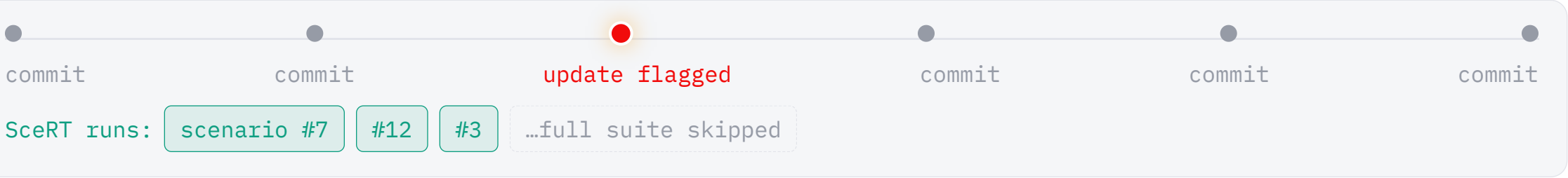
CONTRIBUTION 3 · MAINTAIN

In progress · targeting ICSE 2027

SceRT: keeping up with change

Self-driving software changes **many times a day**, but full re-tests cost tens of CPU-hours — so regressions slip through.

SceRT predicts **which** scenarios a change is likely to break, and runs those first, within a fixed budget.



NEURAL: can't trace by hand

Learn which tests a change affects, from history.

RANDOM: not pass/fail

A new metric that handles **probabilities** (a scenario may crash 30% of the time).

EQUAL COST: per test

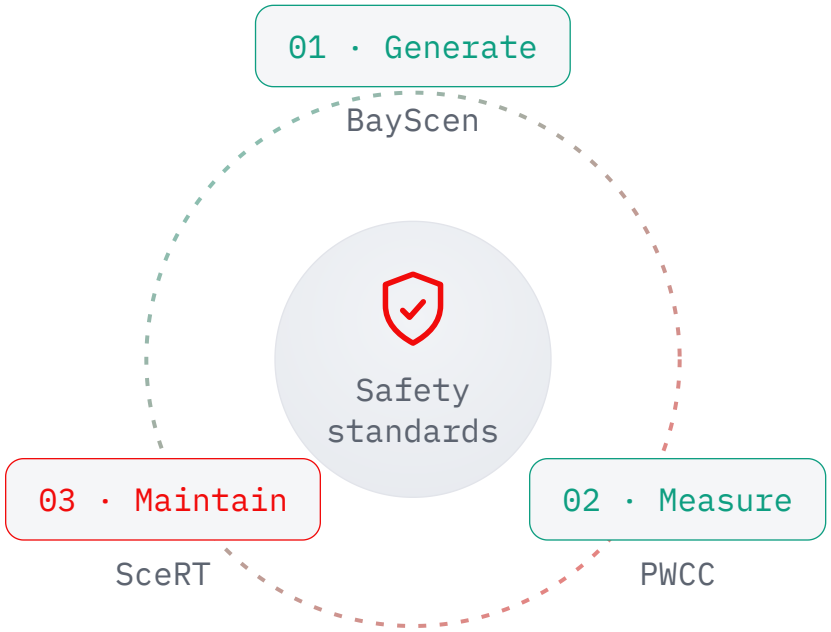
Goal becomes **most failures found per minute**.

Open benchmark · 252 versions × 13 scenario types × 10 repeats ≈ 32,760 simulated runs

01 GENERATE → 02 MEASURE → 03 MAINTAIN SceRT

THE BIG PICTURE

One continuous safety loop



THREE SHIFTS IN MINDSET

1 From raw parameters → to **capabilities**.

2 From "how many tests" → to **real-world risk covered**.

3 From a one-time test → to **living assurance**.

All grounded in safety standards (SOTIF / ISO 34503) → results become **certification-ready evidence**, not just numbers.

WHERE I AM TODAY

Progress & timeline



JUL 2024

PhD start

2025

Vision paper published: ISSRE Doctoral Symposium

UNDER REVIEW

BayScen: ISSRE 2026 · **PWCC**: ASE 2026

IN PROGRESS

SceRT: targeting ICSE 2027 · open benchmark release

MID-2027

Anticipated thesis defense

published · under review · in progress

TAKEAWAY

The question isn't how many crashes a test finds, but whether we test the right risks, prove we've tested enough, and keep it up as the system evolves.

Generate · Measure · Maintain

Thank you. Questions?

Borhaneddine Hamadou · CEA List · Université Paris-Saclay
borhaneddine.hamadou@cea.fr